

A01 Understanding Development Environments and Tools for Edge AI and IoT

Martin Demel, Maira Tanweer Chachar, Jenner Ramos, Ryan Yaunch

Department of Science, Technology, Engineering & Math, Houston
Community College

ITAI-3377-A.I. at the Edge & IIOT Env-RT-17181

Professor Sitaram Ayyagari

January 21st, 2026.

Introduction

Edge AI and IoT systems combine connected devices with on-device intelligence to reduce latency, lower bandwidth costs, and keep sensitive data closer to where it is produced. Building these solutions typically involves collecting sensor data, training and optimizing machine learning models, deploying inference on constrained hardware, and integrating devices with gateways and cloud services. We have summarized in this report 6 common tools used throughout that workflow, explaining what each tool is, why it is used, typical Edge AI/IoT use cases, and a practical example for each.

Body

1. Visual Studio Code (VS Code)

Description

Visual Studio Code is a cross platform editor for coding that can be extended into a full development environment. It provides support for many programming languages and offers features such as code auto completion, deep debugging, Git integration, and an integrated terminal. A major advantage for Edge AI/IoT work is its remote development support, which enables developers to work directly on edge devices or in containers.

Purpose

VS Code is popular because it supports multi-language projects, firmware, ML, and gateways, in a single workspace and enables fast debugging on real hardware, which is essential when performance and device-specific constraints matter.

Typical Use Cases

- Writing and debugging embedded firmware in C/C++.
- Developing Python inference scripts on embedded Linux.
- Remote development on devices, including SSH/containers.

Illustrative Example



2. Node.js

Description

Node.js is a JavaScript runtime for building real-time services, APIs, servers, and command-line tools. Its event driven, non-blocking I/O model makes it well-suited for applications that handle many simultaneous connections and continuous data streams.

Purpose

In IoT, Node.js often powers gateways and backend logic that must reliably transfer data between sensors, edge devices, dashboards, and cloud endpoints in real time.

Typical Use Cases

- IoT gateway software for collecting, buffering, and forwarding telemetry.
- Real-time dashboards with WebSockets/streaming updates.
- Protocol bridging, such as local device protocols to HTTP/MQTT.
- Device provisioning and configuration services.

Illustrative Example



3. Edge Impulse CLI

Description

Edge Impulse CLI is a command-line toolset that supports edge ML workflows, especially for sensor-based projects. It helps connect devices, forward live sensor data for ingestion, upload datasets from local files, and run trained models on hardware for validation.

Purpose

The CLI reduces friction in the most time-sensitive part of edge ML: rapid iteration between data collection, training, and on-device testing. It also improves repeatability by enabling workflows to be scripted and standardized.

Typical Use Cases

- Streaming sensor data from devices into a dataset pipeline.
- Bulk uploading existing data, including audio, images, and sensor logs.
- Automating data collection and regression checks.

Illustrative Example



4. TensorFlow and TensorFlow Lite

Description

TensorFlow is a machine learning framework that is great for building, training, and evaluating models. TensorFlow Lite is a deployment-focused runtime and toolchain for efficient inference on edge devices. A common workflow is to train with TensorFlow, then convert and optimize the model for TensorFlow Lite, often using quantization to reduce size and improve speed.

Purpose

Together, they provide an end-to-end path from model development to edge deployment, balancing accuracy during training with constraints such as memory, latency, and power during inference.

Typical Use Cases

- On-device audio classification and keyword spotting.
- Sensor anomaly detection, including vibration, temperature, and power.
- Edge vision inference, such as classification or detection.

Illustrative Example



5. Google Colab

Description

Google Colab is a cloud-based notebook environment for running Python code. It supports interactive development (code, output, and explanations in one place) and provides access to accelerators such as GPUs, making it well-suited for ML experimentation and training.

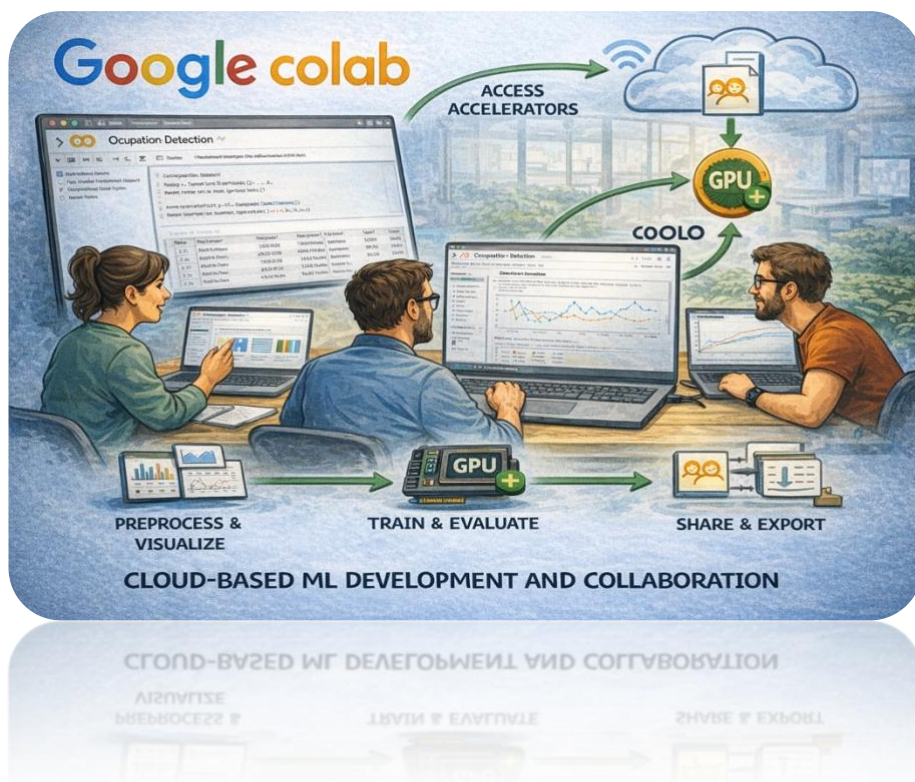
Purpose

Google Colab simplifies ML development by removing local setup barriers and enabling easy sharing of reproducible notebooks that document preprocessing, training, evaluation, and export.

Typical Use Cases

- Data preprocessing and visualization for sensor datasets.
- Model training and evaluation experiments.
- Collaboration and onboarding via shared notebooks

Illustrative Example



6. Generative AI Coding Tools (Copilot and Codex)

Description

Generative AI coding tools offer AI-assisted code completion and chat-based support for writing, understanding, testing, and refactoring code. They can suggest implementations, explain unfamiliar code, generate test cases, and help with debugging.

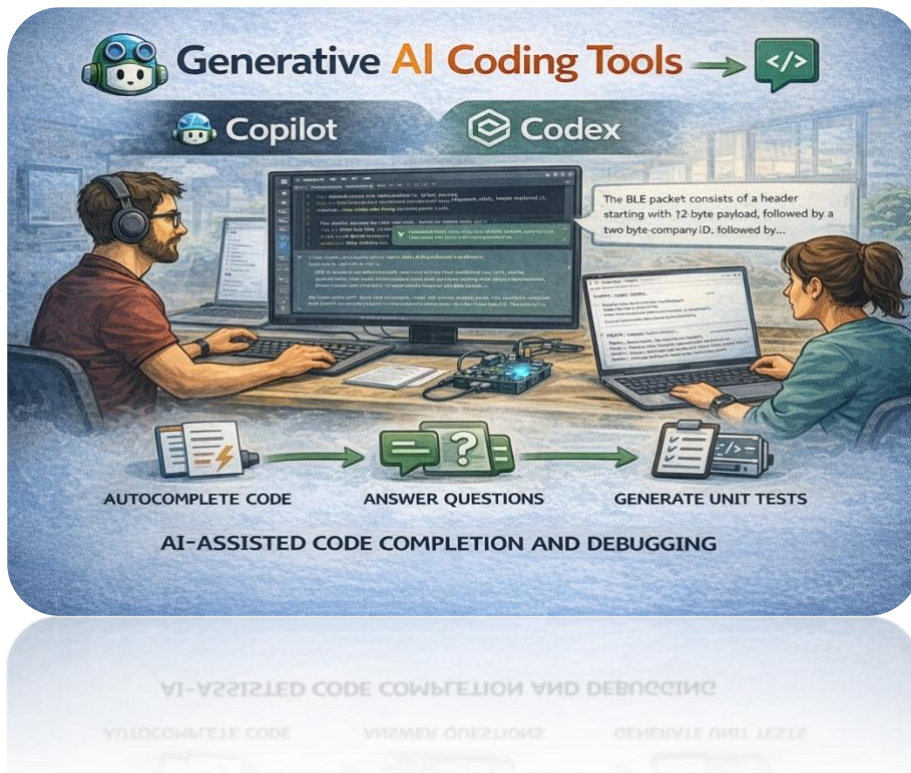
Purpose

These tools help developers move faster through repetitive tasks and reduce time, while also supporting quality through test generation and review assistance. Because edge systems are safety and reliability sensitive, outputs must still be validated through testing and on-device verification.

Typical Use Cases

- Explaining errors and suggesting debugging steps.
- Producing unit tests for packet-decoding or preprocessing code.
- Drafting firmware utilities, parsers, buffers, and state machines.

Illustrative Example



Conclusion

Edge AI and IoT development depends on a coordinated toolchain, a flexible development environment for multi-language code and remote debugging, a runtime for real-time gateway services, CLI tooling to accelerate edge ML iteration, ML frameworks that support training and optimized on-device inference, a cloud notebook platform for reproducible collaboration, and AI coding assistants that accelerate implementation and testing. Together, these tools enable teams to build systems that are accurate, responsive, deployable on constrained hardware, and maintainable over time.